Chetanya
1024030027
3C35

Question 1:

Write a Java program to create a generic method that calculates the sum of an array of numeric values. Use a bounded type parameter such as T extends Number and the doubleValue( ) method to calculate the sum. Test the method using arrays of integers, floats, and doubles.

Code:

```
public class Q1 {

    public static <T extends Number> double sum(T[] arr) {
        double total = 0;
        for (T x : arr) {
            total += x.doubleValue();
        }
        return total;
    }

    public static void main(String[] args) {
        Integer[] ints = {10, 20, 30, 40, 50};
        Float[] floats = {1.5f, 2.5f, 3.5f, 4.5f};
        Double[] doubles = {2.25, 3.75, 10.5, 6.0};

        System.out.println("Sum of integers: " + sum(ints));
        System.out.println("Sum of floats: " + sum(floats));
        System.out.println("Sum of doubles: " + sum(doubles));
    }
}
```

Output:

```
Sum of integers: 150.0
Sum of floats: 12.0
Sum of doubles: 22.5
```

Question 2:

Write a Java program that stores a student's email address in an Optional<String> object. Perform the following operations:

1. Create an Optional containing an email address.
2. Create an empty Optional.
3. Display the email using ifPresent( ).
4. Use orElse( ) to provide a default email address.
5. Use orElseThrow( ) to handle a missing email value.

Code:

```
import java.util.Optional;

public class Q2 {
   public static void main(String[] args) {
      Optional<String> email = Optional.of("chetanya@thapar.edu");
      Optional<String> noEmail = Optional.empty();

      System.out.print("Email using ifPresent: ");
      email.ifPresent(e -> System.out.println(e));
      noEmail.ifPresent(e -> System.out.println("This will not print"));

      String e1 = email.orElse("default@thapar.edu");
      String e2 = noEmail.orElse("default@thapar.edu");
      System.out.println("orElse on present email: " + e1);
      System.out.println("orElse on empty email: " + e2);

      System.out.println("orElseThrow on present email: " + email.orElseThrow());
      try {
         String e3 = noEmail.orElseThrow(() -> new IllegalArgumentException("Email not found
for student"));
         System.out.println(e3);
      } catch (IllegalArgumentException ex) {
         System.out.println("Exception caught: " + ex.getMessage());
      }
   }
}
```

Output:

```
Email using ifPresent: chetanya@thapar.edu
orElse on present email: chetanya@thapar.edu
orElse on empty email: default@thapar.edu
orElseThrow on present email: chetanya@thapar.edu
Exception caught: Email not found for student
```

Question 3:

Create a generic class Result<T extends Number> to store marks of a student. The program should:
1. Store marks using a generic type.
2. Calculate the percentage or average using numeric conversion methods.
3. Determine whether the student has passed based on a specified threshold.
4. Display the result using Optional if the marks are available.
5. Use Objects.requireNonNull( ) to validate required inputs.

Code:

```
import java.util.Objects;
import java.util.Optional;

class Result<T extends Number> {
```

```
private String name;
private T[] marks;
private double maxMarks;

Result(String name, T[] marks, double maxMarks) {
   this.name = Objects.requireNonNull(name, "Student name cannot be null");
   this.marks = marks;
   this.maxMarks = maxMarks;
}

Optional<T[]> getMarks() {
   return Optional.ofNullable(marks);
}

double total() {
   double sum = 0;
   for (T m : marks) {
      sum += m.doubleValue();
   }
   return sum;
}

double average() {
   return total() / marks.length;
}

double percentage() {
   return total() / (maxMarks * marks.length) * 100;
}

boolean isPassed(double threshold) {
   return percentage() >= threshold;
}

void display(double threshold) {
   System.out.println("Student: " + name);
   if (getMarks().isPresent()) {
      System.out.print("Marks: ");
      for (T m : marks) {
         System.out.print(m + " ");
      }
      System.out.println();
      System.out.println("Total: " + total());
      System.out.printf("Average: %.2f%n", average());
      System.out.printf("Percentage: %.2f%%%n", percentage());
      System.out.println("Status: " + (isPassed(threshold) ? "PASS" : "FAIL"));
   } else {
```

```
            System.out.println("Marks not available");
        }
        System.out.println();
    }
}

public class Q3 {
    public static void main(String[] args) {
        double threshold = 40;

        Integer[] m1 = {78, 85, 66, 90, 72};
        Result<Integer> r1 = new Result<>("Chetanya", m1, 100);
        r1.display(threshold);

        Double[] m2 = {32.5, 28.0, 41.5, 35.0};
        Result<Double> r2 = new Result<>("Rahul", m2, 100);
        r2.display(threshold);

        Result<Integer> r3 = new Result<>("Aman", null, 100);
        r3.display(threshold);

        try {
            Result<Integer> r4 = new Result<>(null, m1, 100);
        } catch (NullPointerException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

```
Student: Chetanya
Marks: 78 85 66 90 72
Total: 391.0
Average: 78.20
Percentage: 78.20%
Status: PASS

Student: Rahul
Marks: 32.5 28.0 41.5 35.0
Total: 137.0
Average: 34.25
Percentage: 34.25%
Status: FAIL

Student: Aman
Marks not available
```

Error: Student name cannot be null

Question 4:

Write a Java program to manage an event using LocalDate and LocalTime. The program should:
1. Accept an event name, event date, and start time.
2. Display the event details.
3. Check whether the event date is in the future, today, or in the past.
4. Calculate the number of days remaining until the event.
5. Calculate the event's end time if the event lasts for two hours.
6. Use Optional<String> to handle an optional venue name and display "Venue not specified" when it is absent.

Code:

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.temporal.ChronoUnit;
import java.util.Optional;

class Event {
    String name;
    LocalDate date;
    LocalTime startTime;
    Optional<String> venue;

    Event(String name, LocalDate date, LocalTime startTime, String venue) {
        this.name = name;
        this.date = date;
        this.startTime = startTime;
        this.venue = Optional.ofNullable(venue);
    }

    void showDetails() {
        LocalDate today = LocalDate.now();
        System.out.println("Event Name: " + name);
        System.out.println("Event Date: " + date);
        System.out.println("Start Time: " + startTime);
        System.out.println("End Time: " + startTime.plusHours(2));
        System.out.println("Venue: " + venue.orElse("Venue not specified"));

        if (date.isAfter(today)) {
            System.out.println("Status: Event is in the future");
            System.out.println("Days remaining: " + ChronoUnit.DAYS.between(today, date));
        } else if (date.isEqual(today)) {
            System.out.println("Status: Event is today");
            System.out.println("Days remaining: 0");
        } else {
```

```
            System.out.println("Status: Event is in the past");
            System.out.println("Days since event: " + ChronoUnit.DAYS.between(date, today));
        }
        System.out.println();
    }
}

public class Q4 {
    public static void main(String[] args) {
        System.out.println("Today's Date: " + LocalDate.now());
        System.out.println();

        Event e1 = new Event("Tech Fest", LocalDate.of(2026, 12, 15), LocalTime.of(10, 30), "Main
Auditorium");
        Event e2 = new Event("Coding Contest", LocalDate.now(), LocalTime.of(14, 0), null);
        Event e3 = new Event("Freshers Party", LocalDate.of(2026, 8, 20), LocalTime.of(18, 0),
"Open Air Theatre");

        e1.showDetails();
        e2.showDetails();
        e3.showDetails();
    }
}
```

Output:

```
Today's Date: 2026-10-26

Event Name: Tech Fest
Event Date: 2026-12-15
Start Time: 10:30
End Time: 12:30
Venue: Main Auditorium
Status: Event is in the future
Days remaining: 50

Event Name: Coding Contest
Event Date: 2026-10-26
Start Time: 14:00
End Time: 16:00
Venue: Venue not specified
Status: Event is today
Days remaining: 0

Event Name: Freshers Party
Event Date: 2026-08-20
Start Time: 18:00
End Time: 20:00
```

Venue: Open Air Theatre
Status: Event is in the past
Days since event: 67

Question 5:

Develop a generic utility class containing the following methods:
1. isEqual(T a, T b) — checks whether two objects are equal.
2. getDefault(Optional<T> value, T defaultValue) — returns the contained value or a default.
3. max(T a, T b) — returns the larger of two comparable objects.
4. isNull(Object obj) — checks whether an object is null using the Objects class. Demonstrate the methods using strings, integers, and other suitable objects.

Code:

```
import java.time.LocalDate;
import java.util.Objects;
import java.util.Optional;

class Utility {

   public static <T> boolean isEqual(T a, T b) {
      return Objects.equals(a, b);
   }

   public static <T> T getDefault(Optional<T> value, T defaultValue) {
      return value.orElse(defaultValue);
   }

   public static <T extends Comparable<T>> T max(T a, T b) {
      return a.compareTo(b) >= 0 ? a : b;
   }

   public static boolean isNull(Object obj) {
      return Objects.isNull(obj);
   }
}

public class Q5 {
   public static void main(String[] args) {
      System.out.println("isEqual(\"java\", \"java\"): " + Utility.isEqual("java", "java"));
      System.out.println("isEqual(\"java\", \"python\"): " + Utility.isEqual("java", "python"));
      System.out.println("isEqual(25, 25): " + Utility.isEqual(25, 25));
      System.out.println("isEqual(10.5, 20.5): " + Utility.isEqual(10.5, 20.5));
      System.out.println();

      Optional<String> name = Optional.of("Chetanya");
      Optional<String> empty = Optional.empty();
      Optional<Integer> num = Optional.empty();
```

```
        System.out.println("getDefault(Optional[Chetanya], \"Guest\"): " + Utility.getDefault(name,
"Guest"));
        System.out.println("getDefault(Optional.empty, \"Guest\"): " + Utility.getDefault(empty,
"Guest"));
        System.out.println("getDefault(Optional.empty, 0): " + Utility.getDefault(num, 0));
        System.out.println();

        System.out.println("max(\"apple\", \"mango\"): " + Utility.max("apple", "mango"));
        System.out.println("max(45, 78): " + Utility.max(45, 78));
        System.out.println("max(3.14, 2.71): " + Utility.max(3.14, 2.71));
        LocalDate d1 = LocalDate.of(2026, 1, 26);
        LocalDate d2 = LocalDate.of(2026, 8, 15);
        System.out.println("max(" + d1 + ", " + d2 + "): " + Utility.max(d1, d2));
        System.out.println();

        String s = null;
        System.out.println("isNull(null): " + Utility.isNull(s));
        System.out.println("isNull(\"hello\"): " + Utility.isNull("hello"));
        System.out.println("isNull(100): " + Utility.isNull(100));
    }
}
```

Output:

```
isEqual("java", "java"): true
isEqual("java", "python"): false
isEqual(25, 25): true
isEqual(10.5, 20.5): false

getDefault(Optional[Chetanya], "Guest"): Chetanya
getDefault(Optional.empty, "Guest"): Guest
getDefault(Optional.empty, 0): 0

max("apple", "mango"): mango
max(45, 78): 78
max(3.14, 2.71): 3.14
max(2026-01-26, 2026-08-15): 2026-08-15

isNull(null): true
isNull("hello"): false
isNull(100): false
```